Team 1 — Cognitive Autonomy: Technical Report

VENTEK Autonomous Welding Project · Domain 1 · Due Sunday 18 October 2026 · Baseline target 25 October 2026

Status: draft for team review. All JSON values are **MOCK** illustrations, not measurements. Items marked **[AGREE]** need cross-team agreement. Items marked **[ASSUMPTION]** are unverified and must be tuned or confirmed.

1. Objective and scope

Core question: given the current state of the welding cell, what should the system do next?

Team 1 is the supervisory decision layer. It reads results from perception (Team 5), path planning (Team 6), cell orchestration (Team 2), process monitoring (Team 3) and adaptive control (Team 4), and emits one decision at a time: scan, plan, weld the next joint, rotate the fixture, replan, pause, stop, or ask a human. It does not detect joints, plan trajectories, drive hardware, or correct the path in real time. Those belong to the other teams.

In scope for the 10-day baseline

- A deterministic state machine with explicit transition rules.
- A rule-based decision engine that chooses the next joint and fixture move.
- Input validation (schema, freshness, validity flags) and failure handling.
- A mock-input test harness and a decision log.

Out of scope for now (future phase): reinforcement learning, a trained policy, a digital-twin training environment, and Jetson deployment. Following the guide, advanced AI is added only after the baseline is validated and a measurable limitation is shown (section 11).

2. System states and transition rules

State	Meaning
IDLE	No job loaded; waiting for job.json

State	Meaning
SCANNING	Team 5 is scanning and producing scene.json
PLANNING	Team 6 is producing trajectory results for the requested joints
VALIDATING	Team 1 checks scene and trajectory consistency and builds a weld sequence
READY	Sequence approved; waiting for the cell to be ready for the next command
EXECUTING	A command (rotate fixture or weld a joint) is in progress at Team 2
REPLANNING	A deviation or failure requires a new scan, plan or sequence
PAUSED	Execution halted but recoverable (resume or human decision)
FAULT	Safe stop; needs human action
COMPLETE	All weldable joints done; report generated

Normal flow:

```
IDLE → SCANNING → PLANNING → VALIDATING → READY → EXECUTING → READY (next
joint) … → COMPLETE
Any active state → PAUSED or FAULT on a guard failure.  EXECUTING →
REPLANNING → PLANNING.
```

#	From	Event / guard	To	Decision emitted
T1	IDLE	valid job.json received	SCANNING	request_scan
T2	SCANNING	scene valid, at least 1 joint, every joint confidence ≥ C_min	PLANNING	request_plan
T3	SCANNING	scene invalid, empty, or low confidence, retries < N_retry	SCANNING	request_scan (retry)

#	From	Event / guard	To	Decision emitted
T4	SCANNING	retries exhausted	PAUSED	request_human
T5	PLANNING	every joint has at least one fully valid option	VALIDATING	none
T6	PLANNING	some joints have no valid option at any fixture angle	VALIDATING	none; those joints flagged blocked, human notified
T7	PLANNING	no joint has any valid option	PAUSED	request_human
T8	VALIDATING	sequence built, all checks pass	READY	none
T9	READY	cell fresh, robot idle, no e-stop, clamps locked, PLC ok	EXECUTING	rotate_fixture or execute_weld
T10	EXECUTING	last_command_id matches, status completed, joints remain	READY	none
T11	EXECUTING	completed, no joints remain	COMPLETE	report_complete
T12	EXECUTING	correction says corrected and within limits	EXECUTING	none; logged
T13	EXECUTING	correction says replan, or deviation > D_replan	REPLANNING	pause, then request_plan
T14	EXECUTING	process anomaly active, or quality below minimum	PAUSED	pause
T15	EXECUTING	command timeout	PAUSED (first), FAULT (second)	pause, then stop

#	From	Event / guard	To	Decision emitted
T16	any	e-stop, PLC fault, robot fault, collision flag	FAULT	stop
T17	any active	any required input older than T_stale	PAUSED	pause
T18	PAUSED	cause cleared and human confirms	previous safe state	resume
T19	REPLANNING	new valid trajectory results	VALIDATING	none

Thresholds [ASSUMPTION], held in a YAML config and tuned during testing: C_min = 0.6, N_retry = 2, T_stale = 0.5 s for cell_state, process_status and correction, D_replan = 0.002 m (or the job's max_deviation_m), command timeout = 2 × estimated time + 10 s, minimum quality = job's min_quality_score.

3. Approach and alternatives

Workflow

1. Load the job (material, process, priorities).
2. Request a scan; accept scene.json only if valid and confident.
3. Request trajectory results from Team 6 for each joint, at the current and alternative fixture angles.
4. Build a weld sequence with the rule-based selector (section 9).
5. Issue one command at a time and wait for Team 2 to confirm completion, not merely receipt.
6. Monitor Team 3 and Team 4 feedback; continue, pause, replan or stop by the transition rules.
7. On completion, write the weld report and decision log.

Option	Strengths	Weaknesses	Verdict
State machine + rules	Transparent, testable, easy to audit for safety	Sequence heuristics are hand-written	**Chosen baseline**

Option	Strengths	Weaknesses	Verdict
Behaviour tree	Modular, reactive	More tooling	Consider if the state machine grows unwieldy
Search / optimisation (exhaustive or beam) for sequencing	Optimal for small joint counts	Needs a cost model	**Second step, before RL**
PDDL task planner	Formal	Heavy setup for a small problem	Not needed
Reinforcement learning (PPO)	May learn trade-offs in complex parts	Needs simulator and reward design; hard to certify	Future, only if justified
LLM reasoning	Flexible	Non-deterministic; unsuitable for safety-critical control	Not used for control decisions

4. Technology stack and why

Component	Choice	Why it is needed
Language	Python 3 (version to match the ROS 2 distribution in use)	Fast to build and test; the team and the other domains already use Python
Middleware	ROS 2 Humble with rclpy	Teams 2, 4, 5 and 6 plan to use ROS 2, so topics are the common interface
State machine	Plain Python Enum plus a transition table loaded from YAML (the `transitions` library is optional)	Every rule stays visible and unit-testable; no heavy dependency
Message validation	pydantic or jsonschema	Rejects malformed or incomplete inputs before they reach the logic
Configuration	YAML	Thresholds can be tuned without changing code

Component	Choice	Why it is needed
Testing	pytest with JSON scenario fixtures	Repeatable normal and failure scenarios; gives the evidence needed
Logging and replay	JSON Lines decision log plus rosbag2	Every decision is traceable to the inputs used; runs can be replayed
Mock environment	Python mock publishers for each team's message	Lets Team 1 work before other teams deliver; Gazebo and MoveIt 2 are reached only through Teams 2 and 6
Hardware	None for the baseline; a laptop or Docker with ROS 2	Team 1 owns no physical device
Later phase	Gymnasium, Stable-Baselines3 (PPO), PyTorch, PyBullet or MoveIt 2 for collision checks, ONNX or TensorRT on an NVIDIA Jetson	Only if section 11 criteria are met

All versions and library choices above are proposals; none have been tested in this project yet.

5. Inputs: what Team 1 needs, in what form

Common conventions [AGREE]: valid JSON; SI units (metres, radians, seconds); timestamps in ISO 8601 UTC with milliseconds; quaternions ordered [x, y, z, w]; positions in the frame `cell_base` (robot base frame) unless stated; a `header` on every message.

Header field	Type	Meaning
schema_version	string	Interface version, e.g. 0.1
producer	string	team1 … team6
stamp	string	ISO 8601 UTC time the data was produced
seq	integer	Increasing counter; detects gaps and repeats
frame_id	string	Coordinate frame of any positions

Header field	Type	Meaning
valid	boolean	False means the producer itself does not trust this message

5.1 scene.json — from Team 5 (Vision), once per scan

```
{
  "header": {"schema_version": "0.1", "producer": "team5", "stamp": "2026-10-
20T10:15:30.120Z", "seq": 1, "frame_id": "cell_base", "valid": true},
  "scene_id": "MOCK-scene-001",
  "objects": [{"id": "part_1", "class": "part", "confidence": 0.93}, {"id":
"clamp_1", "class": "clamp", "confidence": 0.88}],
  "joints": [
    {"joint_id": "J1", "type": "fillet", "start_m": [0.10, 0.20, 0.05],
"end_m": [0.34, 0.20, 0.05], "length_m": 0.24, "normal": [0, 0, 1], "source":
"scan", "confidence": 0.91},
    {"joint_id": "J2", "type": "butt", "start_m": [0.34, 0.20, 0.05],
"end_m": [0.34, 0.325, 0.05], "length_m": 0.125, "normal": [0, 0, 1],
"source": "scan_cad_confirmed", "confidence": 0.84}
  ]
}
```

Team 1 uses: `joint_id` (string, required), `type` (fillet, butt, lap; required), `start_m` / `end_m` (metres, required, used to find neighbouring joints), `length_m` (required), `confidence` (0 to 1, required), `source` (optional). **[AGREE]** how joint coordinates are expressed once the fixture rotates.

5.2 trajectory.json — from Team 6 (Path planning), per planning request


```
{
  "header": {"schema_version": "0.1", "producer": "team6", "stamp": "2026-10-
20T10:15:45.800Z", "seq": 3, "frame_id": "cell_base", "valid": true},
  "scene_id": "MOCK-scene-001",
  "results": [
    {"joint_id": "J1", "fixture_angle_rad": 0.0, "reachable": true,
"collision_free": true, "singularity_free": true, "est_time_s": 42.0,
"planning_time_ms": 180, "trajectory_ref": "traj_J1_a0.json"},
    {"joint_id": "J2", "fixture_angle_rad": 0.0, "reachable": false,
"collision_free": null, "singularity_free": null, "est_time_s": null,
"failure_reason": "ik_no_solution"},
    {"joint_id": "J2", "fixture_angle_rad": 0.52, "reachable": true,
"collision_free": true, "singularity_free": true, "est_time_s": 47.0,
"planning_time_ms": 210, "trajectory_ref": "traj_J2_a52.json"}
  ]
}
```


Team 1 needs only the flags, time and a reference to the full trajectory; Team 2 consumes the trajectory itself. A result is usable only if reachable, collision_free and singularity_free are all true. `failure_reason` is optional text.

5.3 cell_state.json — from Team 2 (Orchestration), live


```
{
  "header": {"schema_version": "0.1", "producer": "team2", "stamp": "2026-10-
20T10:16:01.900Z", "seq": 88, "frame_id": "cell_base", "valid": true},
  "robot_state": "idle",
  "fixture_angle_rad": 0.0,
  "clamps_locked": true,
  "plc_ok": true,
  "estop_active": false,
  "last_command_id": "cmd-0006",
  "last_command_status": "completed",
  "available_actions": ["rotate_fixture", "execute_weld", "pause"],
  "faults": []
}
```


`robot_state`: idle, moving, welding or error. `last_command_status`: accepted, in_progress, completed or failed. This is how Team 1 confirms a command finished rather than merely sent. **[AGREE]** that Team 2 echoes the command ID from decision.json.

5.4 process_status.json — from Team 3 (Multimodal AI), live

```
{
  "header": {"schema_version": "0.1", "producer": "team3", "stamp": "2026-10-
20T10:16:01.700Z", "seq": 210, "frame_id": "cell_base", "valid": true},
  "quality_score": 0.87,
  "defect_probability": 0.03,
  "anomaly": {"active": false, "type": null},
  "sensor_status": {"current_voltage": "ok", "temperature": "ok"},
  "uncertainty": 0.05
}
```

Quality and defect probability are unitless (0 to 1). Team 1 uses summarised outputs only, never raw signals. **Optional in the baseline**, because Team 3's first prediction task is not yet defined. **[AGREE]** what quality_score is evaluated against.

5.5 correction.json — from Team 4 (Adaptive control), live during welding

```
{
  "header": {"schema_version": "0.1", "producer": "team4", "stamp": "2026-10-
20T10:16:01.800Z", "seq": 95, "frame_id": "cell_base", "valid": true},
  "joint_id": "J1",
  "deviation_m": 0.0006,
  "correction_applied_m": 0.0005,
  "within_limits": true,
  "action_requested": "none",
  "confidence": 0.9
}
```

`action_requested`: none, corrected, replan, pause or human. Small deviations stay inside Team 4; only replan, pause or human requests, or a deviation above D_replan, change Team 1's state.

5.6 job.json — from the customer or operator, once per job

```
{
  "job_id": "MOCK-job-001", "process": "MIG", "material": "mild_steel",
"thickness_m": 0.003,
  "priority": ["quality", "time", "cost"], "min_quality_score": 0.85,
"max_deviation_m": 0.002
}
```

6. Output: decision.json

Producer: Team 1. **Consumers:** Team 2 (executes), Team 6 (replan and plan requests), Team 5 (scan requests), Teams 3 and 4 (logging and mode awareness).


```
{
  "header": {"schema_version": "0.1", "producer": "team1", "stamp": "2026-10-20T10:16:02.400Z", "seq": 7, "frame_id": "cell_base", "valid": true},
  "decision_id": "cmd-0007",
  "state": "READY",
  "next_state": "EXECUTING",
  "action": "rotate_fixture",
  "target": {"joint_id": "J2", "fixture_angle_rad": 0.52, "trajectory_ref": null},
  "reason_code": "JOINT_UNREACHABLE_AT_CURRENT_ANGLE",
  "reason_text": "J2 unreachable at 0.0 rad; reachable and collision-free at 0.52 rad",
  "inputs_used": {"scene_seq": 1, "trajectory_seq": 3, "cell_state_seq": 88, "process_seq": 210, "correction_seq": null},
  "confidence": 0.9,
  "requires_human": false,
  "timeout_s": 60.0
}
```


Field	Type	Meaning
decision_id	string	Unique command ID; Team 2 echoes it as last_command_id
state / next_state	string	Team 1's state before and after this decision
action	string	request_scan, request_plan, rotate_fixture, execute_weld, replan, pause, resume, stop, request_human, report_complete, no_op
target	object	joint_id, fixture_angle_rad (rad), trajectory_ref; fields not relevant to the action are null
reason_code / reason_text	string	Machine-readable and human-readable reason; enables audit
inputs_used	object	seq number of each input behind the decision
confidence	number	0 to 1; lowest confidence among the inputs used

Field	Type	Meaning
requires_human	boolean	True means no further automatic action until a person responds
timeout_s	number	Seconds Team 2 has to complete; Team 1 pauses when exceeded

[AGREE] transport for the baseline: JSON strings on ROS 2 topics (for example `/team1/decision`, reliable QoS) to move quickly, with typed custom messages later.

7. Interfaces and cross-team agreements

Provider	Consumer	Message	Needs agreement
Team 5	Team 1	scene.json	Frame, joint-coordinate convention after fixture rotation, confidence meaning
Team 6	Team 1	trajectory.json	Which fixture angles are evaluated; response time per request
Team 2	Team 1	cell_state.json	Command-ID echo and status values; update rate
Team 3	Team 1	process_status.json	Which quality target is predicted; update rate; uncertainty meaning
Team 4	Team 1	correction.json	Replan threshold; who owns the limit on corrections
Operator	Team 1	job.json	Source of the job data
Team 1	Teams 2, 5, 6	decision.json	Action list and ID handling

8. Failure handling

Failure	Detection	Response
Failed detection	scene valid=false, no joints, or confidence < C_min	Retry scan up to N_retry, then pause and request a human

Failure	Detection	Response
Unreachable joint	No fully valid option at any angle	Mark joint blocked; continue with the others; flag human. If all blocked, pause
Collision	Team 6 collision_free=false, or a collision fault from Team 2	Never schedule it; during execution stop and go to FAULT
Stale sensor data	Header age > T_stale or seq not advancing	Pause; resume only when data is fresh and a human confirms
Component fault	e-stop, PLC fault, robot error, valid=false	FAULT and stop command
Large deviation	deviation > D_replan or replan requested	Pause, replan, re-validate
Low quality or anomaly	Team 3 anomaly active or quality below minimum	Pause and request human
Command not confirmed	No completed status within timeout_s	Pause; second timeout goes to FAULT

9. Rule-based baseline

One pass per cycle, deterministic:

```
1. Validate every input (schema, valid flag, age < T_stale). Failure → rules
T16 / T17.
2. Run the transition table (section 2).
3. In READY:
   remaining = joints not welded and not blocked
   if remaining is empty → report_complete
   usable = joints in remaining with a fully valid option at the current
fixture angle
   if usable is not empty:
       prefer joints that do NOT share an endpoint (within 10 mm) with the
last welded joint
       among those, choose the shortest estimated time        → execute_weld
   else:
       pick the remaining joint with a valid option at the smallest angle
change → rotate_fixture
```

The rule of not welding neighbouring joints back to back follows general practice of spreading heat to limit distortion. **[ASSUMPTION]** confirm the exact sequencing rules with a welding engineer at the company.

10. Scenarios

Normal

- **N1, single angle:** three reachable joints at 0 rad. Expect: scan, plan, then execute_weld on non-adjacent joints first, ending in report_complete.
- **N2, fixture rotation:** J2 unreachable at 0 rad, reachable at 0.52 rad. Expect: weld the reachable joints, then rotate_fixture to 0.52, weld J2, report_complete.
- **N3, small deviation:** Team 4 reports 0.6 mm, action none. Expect: no pause, no state change, event logged.

Failure

- **F1, failed detection:** low-confidence scene three times. Expect: two retries, then request_human and PAUSED.
- **F2, blocked joint:** J3 has no valid option at any angle. Expect: J3 excluded, human flagged, others welded.
- **F3, collision fault:** Team 2 reports a collision during welding. Expect: stop and FAULT.
- **F4, stale data:** cell_state older than 0.5 s during EXECUTING. Expect: pause; no new execute_weld until fresh.

- **F5, large deviation:** deviation 3 mm with D_replan 2 mm. Expect: pause, request_plan, return to VALIDATING.
- **F6, unconfirmed command:** no completed status within the timeout, twice. Expect: pause, then stop and FAULT.

11. Validation and the future role of RL

Test data: JSON fixtures for the nine scenarios above, plus 100 randomised mock parts (3 to 8 joints, random reachability).

Metric	Target [ASSUMPTION]	Evidence
Scenario pass rate	9 of 9	pytest report
Transition coverage	Every row T1–T19 exercised at least once	coverage table
Safety invariant	0 execute_weld decisions with e-stop, stale, or invalid input across 1,000 fault-injected runs	test log
Decision latency	95th percentile under 50 ms	timing log
Schema validity	100 % of outputs valid against schema	validator output
Determinism	Same inputs give same decisions on replay	rosbag2 replay
Sequence quality	Fewer fixture rotations and fewer adjacent back-to-back welds than welding joints in plain list order, over 100 mock parts	comparison table

When RL could help, and the evidence required. For parts with few joints, exhaustive or beam search may already find the best sequence, so RL is only justified if (a) the baseline and search methods are measurably worse than an alternative on a defined metric, (b) the problem has grown beyond what search handles (many joints, uncertain heat or deformation), and (c) a simulator exists whose rules or surrogate models come from Team 3 and Team 4 data. If so, the plan is a Gymnasium environment, PPO via Stable-Baselines3, domain randomisation, comparison against the rule-based and search baselines on randomised parts, and a safety layer so Team 6 collision checks can veto any command. Jetson deployment follows only if latency requires it. Results from simulation must not be presented as results on real welds.

12. Risks and open questions

- Other teams' formats may differ from this proposal; schema v0.1 must be circulated on day 1.
- Team 3 and Team 4 models may not exist yet, so Team 1 must run with mocks and treat their inputs as optional.
- Coordinate frames can change when the fixture rotates; unresolved until Teams 2, 5 and 6 agree.
- There is no real hardware for Team 1, so nothing here is validated on a physical cell.
- All thresholds are guesses until tested.
- The human-intervention channel (how a person confirms resume) is not yet defined.
- Open: command-ID echo, update rates, who owns correction limits, and the quality ground truth.

13. 10-day plan

Day	Task	Owner	Evidence
1 (18 Oct)	Submit report; circulate schema v0.1 to Teams 2–6	Team Lead	Message sent; comments logged
2	Write schemas for the six inputs and decision.json	Member 1	Schema files; validator passes on examples
3	State machine and YAML transition table in code	Team Lead, Member 2	Unit tests for T1–T19
4	Input validator, freshness checks, mock publishers	Member 1, Member 3	Mock messages replay
5	Rule-based sequence selector	Member 2	Output on N1 and N2
6	Scenario fixtures N1–N3, F1–F6 and pytest suite	Member 3	Test report
7	ROS 2 node, topics, JSON Lines decision log	Member 4	Node runs against mocks; log file
8 (25 Oct)	Full baseline demo and metrics table	All	Demo recording; metrics table

Day	Task	Owner	Evidence
9–10	Buffer: adopt real inputs where teams are ready; fix mismatches; document	All	Updated schema; notes

Definition of done: all nine scenarios pass; every transition tested; outputs validate against the schema; the demo runs end to end on mock inputs; open interface questions are documented and sent to the owning teams.

14. References

Proposed documentation to read and cite; links not yet reviewed in depth.

- VENTEK Team Report Guide (project document supplied to teams)
- ROS 2 Humble documentation: https://docs.ros.org/en/humble/
- MoveIt 2: https://moveit.picknik.ai/
- JSON Schema: https://json-schema.org/
- python-transitions: https://github.com/pytransitions/transitions
- Gymnasium: https://gymnasium.farama.org/
- Stable-Baselines3: https://stable-baselines3.readthedocs.io/
- PyBullet: https://pybullet.org/